

06 “ Tests extraction

Exhaustive extraction of the reference browser-extension (“BobaLink”) test suite, fixtures, and test configuration. Source base dir (note the spaces): /Volumes/T7/Projects/Boba/docs/research/browser_extension/Browser Torrent Extension Guide/src

This document drives the test-coverage phases of the implementation plan. Per Boba’s 100%-coverage-across-all-test-types mandate (CONST 11.4.27 / 11.4.25 / 11.4.85), the reference suite (only **unit** + **e2e**) is a STARTING point “ the coverage map and missing-types list below enumerate the gap to 100%.

Files read

File	Lines Type
tests/unit/chrome-mock.ts	196 Mock infra (chrome.* API)
tests/unit/setup.ts	48 Jest setup-after-env
tests/unit/api-client.test.ts	217 Unit (Jest)
tests/unit/bencode.test.ts	252 Unit (Jest)
tests/unit/crypto.test.ts	656 Unit (Vitest , self-contained)
tests/unit/magnet.test.ts	239 Unit (Jest)
tests/unit/queue.test.ts	197 Unit (Jest)
tests/unit/scanner.test.ts	213 Unit (Jest)
tests/unit/torrent-file.test.ts	210 Unit (Jest)
tests/e2e/content.spec.ts	93 E2E (Playwright)
tests/e2e/options.spec.ts	93 E2E (Playwright)
tests/e2e/popup.spec.ts	62 E2E (Playwright)
tests/fixtures/magnets.json	55 Fixture data
jest.config.ts	143 Config (unit)
playwright.config.ts	133 Config (e2e)
TOTAL	2807 “

Test counts: 7 unit .test.ts files (121 it blocks), 3 e2e .spec.ts files (24 test blocks). 1 fixture JSON. 2 configs. Plus 2 mock/setup support files.

CRITICAL DISCREPANCIES FOUND (flag for plan)

1. **Two test runners mixed.** `crypto.test.ts` imports from `vitest` (`import { describe, it, expect, beforeEach } from 'vitest'`); every other unit test uses **Jest** globals. `jest.config.ts` `testMatch` would pick up `crypto.test.ts` but the `vitest` import would fail under Jest. The reference is internally inconsistent – the Boba implementation MUST standardize on ONE runner (likely Vitest, since the Boba frontend/ already uses Vitest).
 2. `crypto.test.ts` **tests a private copy, not the module.** It does NOT import `src/ utils/crypto.ts`; it re-implements `deriveKey/encrypt/decrypt/encryptObject/decryptObject` inline (comment line 27: –mirrors `src/ utils/crypto.ts` –). **This is a BLUFF per CONST 11.4 / 11.4.1** – it can never fail against a no-op stub of the real module because it never calls the real module. Boba™s `crypto` tests MUST import the production `crypto` module.
 3. `setup.ts` **references a wrong path.** Line 14 declares `var chrome: typeof import("../fixtures/chrome-mock").chromeMock` but the mock actually lives at `tests/unit/chrome-mock.ts` (NOT `tests/fixtures/`). `jest.config.ts` `setupFiles` correctly points at `<rootDir>/tests/unit/chrome-mock.ts`, so this is a stale type-only declare global reference – harmless at runtime but wrong.
 4. `playwright.config.ts` **references missing files.** `globalSetup: "../tests/e2e/global-setup.ts"` and `globalTeardown: "../tests/e2e/global-teardown.ts"` are referenced but DO NOT EXIST in `tests/e2e/`. Also `webServer.url: "http://localhost:3000"` is referenced but the build produces a static extension (no dev server on :3000) – likely dead config.
 5. **E2E tests navigate to chrome-extension://test-id/... hardcoded.** `popup/options` specs page `goto("chrome-extension://test-id/...")` with a fixed extension id. Real MV3 extension ids are assigned at load time; these specs would never resolve against a real loaded extension without a fixture that captures the actual id. **These e2e tests are effectively non-functional** as written (no fixture loads the extension and rewrites the URL). They are skeletons, not working e2e. Flag heavily – Boba e2e MUST load the real built extension and resolve the actual id.
 6. **Many –assert true– no-op tests** (anti-bluff violations, see per-file notes): `api-clientAuth` block (`expect(true).toBe(true)`), `queue` auto-processing block (`expect(true).toBe(true)`). These assert nothing user-observable and pass against a no-op stub.
 7. `collectCoverageFrom` **EXCLUDES the entire UI + entrypoints** (`!src/popup, !src/options, !src/content, !src/background, !src/**/index.ts`). So the 80% coverage threshold only applies to pure-logic modules (`parser/api/scanner/shared/ utils`). UI/entrypoint logic has ZERO unit-coverage enforcement in the reference.
-

Per-test-file inventory

tests/unit/chrome-mock.ts (mock infra)

Provides a complete `chrome.*` mock, assigned to `globalThis.chrome` at module load (line 196). Loaded via Jest `setupFiles` BEFORE each test file.

- `MockStorageArea` implements `chrome.storage.StorageArea` â€” backed by an in-memory `Map<string, unknown>`. Implements:
 - `get(keys)` â€” supports `null` (all), `string` (single), `string[]` (subset), and object-keys form; returns a `Promise<Record>`.
 - `set(items)` â€” writes all entries.
 - `remove(keys)` â€” string or array.
 - `clear()` â€” empties the map.
 - `getBytesInUse` â†’ always `Promise.resolve(0)` (stub).
 - `onChanged` â€” no-op listener stubs (`addListener/removeListener/hasListener/hasListeners`).
- `mockRuntime` â€” `id: "test-extension-id"; onInstalled/onStartup/onMessage` no-op listener objects; `sendMessage()` â†’ `Promise.resolve({ success: true })`; `openOptionsPage()` no-op; `getManifest()` â†’ `{ manifest_version: 3, name: "BobaLink", version: "1.0.0" }`; `getURL(path)` â†’ `chrome-extension://test-id/${path}`.
- `mockTabs` â€” `query()` â†’ one fake tab `{ id:1, url:"https://example.com", title:"Example", active:true, windowId:1 }`; `sendMessage()` â†’ `{ success:true }`; `create()` â†’ `{ id:2 }`.
- `mockAlarms` â€” `create()` no-op; `onAlarm` no-op listeners.
- `mockNotifications` â€” `create()` no-op.
- `mockContextMenus` â€” `create()` no-op; `onClicked` no-op listeners.
- `mockAction` â€” `setBadgeText()`, `setBadgeBackgroundColor()` no-ops.
- `mockStorage` â€” `local: new MockStorageArea()`; `onChanged` no-op listeners. (NOTE: no sync, session, or managed areas mocked.)
- `mockCommands` â€” `onCommand` no-op listeners.
- `chromeMock` â€” assembles all the above; assigned to `globalThis.chrome`.

Gap in mock: no `chrome.cookies`, no `chrome.webRequest/declarativeNetRequest`, no `chrome.scripting`, no `chrome.permissions`, no `chrome.storage.sync/session`, no `chrome.downloads`, no error-injection (every method is a happy-path stub returning success). Chaos/error-path unit tests cannot use this mock as-is.

tests/unit/setup.ts (Jest setup-after-env)

- Imports `@testing-library/jest-dom` (extends matchers â€” e.g. `.toBeVisible()`, `.toHaveText()` for DOM assertions in jsdom).
- declare `global { var chrome: typeof import("../fixtures/chrome-mock").chromeMock }` â€” **wrong path** (see discrepancy #3); type-only.
- **Console suppression** (unless `process.env.DEBUG` set): in `beforeAll` replaces `console.log/debug/info/warn` with no-op, restores in `afterAll`. `console.error` deliberately KEPT visible.
- `jest.setTimeout(10000)` â€” 10s default async timeout.

tests/unit/api-client.test.ts (Unit, Jest) â€” BobaAPIClient

Imports BobaAPIClient from ../../src/api/client; NetworkError, ServerError, RateLimitError from ../../src/shared/errors. Mocks global.fetch via jest.fn() (mockFetch). beforeEach: new client ("http://localhost:8080", 5000), mockFetch.mockClear().

- Constructor
 - creates client with base URL â€” getUrl() === "http://localhost:8080".
 - strips trailing slash from URL â€” "http://localhost:8080/" â†’ getUrl() === without slash.
- Auth â€” **BLUFF** â€” both tests end in expect(true).toBe(true):
 - sets auth cookie â€” setAuthCookie("test-sid-123") then asserts true (no observable check).
 - clears auth cookie â€” setAuthCookie("test") then setAuthCookie(null), asserts true.
- Login (mocks fetch Response):
 - returns true on successful login â€” fetch 200 body "Ok." + set-cookie: SID=abc123 â†’ login("admin", "admin") === true.
 - returns false on failed login â€” 200 body "Fails." â†’ false.
 - returns false on HTTP error â€” 401 â†’ false.
 - handles network errors gracefully â€” mockRejectedValueOnce(Error) â†’ false.
- Version check:
 - returns version string on success â€” 200 "v4.6.0" â†’ getVersion() === "v4.6.0".
 - throws NetworkError on timeout â€” fetch rejects after 100ms â†’ getVersion() rejects NetworkError.
- Add torrent from magnet:
 - returns true on success â€” 200 "Ok." â†’ addTorrentFromMagnet(magnet:?xt=urn:btih:1234...5678) === true.
 - throws ServerError on failure â€” 200 "Fails." â†’ rejects ServerError.
- Request method:
 - makes GET request â€” 200 JSON {"version": "v4.6.0"} + content-type json â†’ get<T>("/api/test") deep-equals object.
 - makes POST request with FormData â€” asserts mockFetch called and init.method === "POST". Uses real FormData.
 - retries on server error â€” first 500, then 200 JSON â†’ result equals {ok: true} AND mockFetch called **2 times** (asserts retry logic).
 - throws RateLimitError on 429 â€” 429 + retry-after: 60, called with { retry: false } â†’ rejects ServerError. (NOTE: test name says RateLimitError but asserts ServerError â€” possible inconsistency.)
- Error handling:
 - handles text/plain responses â€” 200 "v4.6.0" text/plain â†’ get<string> === "v4.6.0".
 - handles 204 No Content â€” 204 null body â†’ get<undefined> === undefined.

Covers: request building, trailing-slash normalization, cookie auth (weakly), login flow, version, magnet add, GET/POST, retry-on-5xx, rate-limit/429, content-type branching (json/text/204).

Does NOT cover: real cookie header propagation (the Auth block is a bluff), timeout boundary exactness, concurrent requests, abort-controller cancellation.

tests/unit/bencode.test.ts (Unit, Jest) â€” bencode encode/decode

Imports encode, decode, bytesToHex, hexToBytes, type BencodeValue from ../../src/parser/bencode. Decodes via new TextEncoder().encode(...), asserts via new TextDecoder().decode(...).

- Integers: encode 42â†’i42e, 0â†’i0e, -42â†’i-42e; decode same three; encode(3.14) throws (non-integer).
- Strings: encode â†’`0:`, `hello`â†’`5:hello`, `hello world`â†’`11:hello world`; decode `5:hello`â†’`hello`, `0:``â†’`.
- Lists: encode []â†’le, [1,2,3]â†’li1ei2ei3ee, [1,"hello"]â†’li1e5:helloe; decode leâ†’[], li1ei2ei3eeâ†’[1,2,3].
- Dictionaries: encode {}â†’de; encode {b:2,a:1}â†’d1:ai1e1:bi2ee (**asserts key sorting**); decode deâ†’{}, d1:ai1e1:bi2eeâ†’{a:1,b:2}; decode nested d4:infod4:name5:helloeeâ†’info=={name:"hello"}.
- Nested structures: encode [{a:1},{b:2}]â†’ld1:ai1eed1:bi2eee; decode same.
- Binary data: encode Uint8Array([0,1,2,255]) â†’ prefix 4: (raw bytes). **NOTE line 156 is dead code** (expect(result[0]).toCharCode ? undefined : undefined; â€” a no-op typo, asserts nothing). Decode 5:hello â†’ default typeof === "string"; with {encoding:"binary"} â†’ instanceof Uint8Array.
- Error handling: throws for unterminated integer i42, empty integer ie, unterminated list li42e, unterminated dict d1:ai42e, invalid start byte x, trailing data i42eextra.
- Utility functions: bytesToHex([0,15,255])â†’"000fff"; hexToBytes("000fff")â†’bytes; round-trip inverse on 8 bytes.
- Torrent-like structure: round-trips a dict with announce, announce-list, creation date, nested info{name,piece length,pieces:Uint8Array(20),length}; asserts decoded fields preserved.

Covers: all 4 bencode types, binary mode, key-sorting invariant, hex utils, malformed-input rejection, torrent round-trip. Solid. **Dead-assert line 156 to remove.**

tests/unit/crypto.test.ts (Unit, Vitest â€” self-contained, NOT importing prod) â€” Credential Encryption

â€” i,â€” **BLUFF risk #2.** Re-implements deriveKey/getRandomBytes/toBase64/fromBase64/encrypt/decrypt/encryptObject/decryptObject inline using Web Crypto (crypto.subtle). PBKDF2(SHA-256, 100000 iters)â†’AES-GCM-256, 16-byte salt, 12-byte IV, EncryptedData{ciphertext,iv,salt,version:1} base64. Tests the inline copy, never src/Utils/crypto.ts.

- Roundtrip (11): simple string; password; API key; **different ciphertext each encryption** (iv/salt/ciphertext all differ); preserves version===1; produces valid base64 (fromBase64 not-throw on all 3 fields); Unicode (CJK + emoji + accents); very long (10000 chars); special chars; newlines (\n\r\n); tabs.
- Wrong Password (6): wrong pw rejects; totally-different rejects; empty pw rejects; case variation (Passwordâ€” password) rejects; trailing-space pw rejects; different

passwords â†’ different derived keys (verified indirectly: `exportKey('raw')` returns null because key is non-extractable â€” asserts `raw1 === null`).

- Empty Data (2): encrypt/decrypt "" round-trips; empty input still has non-empty ciphertext (auth tag).
- Large Data (4): 1 KB, 10 KB, 100 KB round-trip; 1 MB round-trip + **perf assert** <5000ms.
- Object Encryption (6): credentials object deep-equal; nested object (server/auth/features) deep-equal; array data (3 trackers) length 3; booleans preserved (true/true/false/false); numerics preserved (incl. float 50.5); null values preserved.
- IV and Salt (4): IV length 12; salt length 16; unique IV per encryption; unique salt per encryption.
- Key Derivation (2): same pw+salt â†’ interoperable keys (encrypt with key1, decrypt with key2); different salts â†’ both keys defined.
- Tamper Detection (3): flip ciphertext byte â†’ decrypt rejects; flip IV byte â†’ rejects; flip salt byte â†’ rejects (GCM auth).
- Performance (3): encrypt short <2000ms (PBKDF2 slow by design); decrypt short <100ms; 10 consecutive encrypt+decrypt round-trip <10000ms.
- Password Edge Cases (4): short pw x; very long pw (1024 chars); Unicode pw; pw with spaces.
- Serialization (3): JSON-serializable; survives JSON round-trip; has all required fields with correct types (ciphertext/iv/salt:string, version:number).

Covers (conceptually): AES-GCM-256 + PBKDF2 credential encryption, tamper detection, IV/salt randomness, perf budgets, object serialization. **The single most thorough file (44 tests).**

BUT it tests a copy â†’ MUST be rewritten to import the real production crypto module so it fails against a no-op stub (CONST Â§11.4.1).

tests/unit/magnet.test.ts (Unit, Jest) â€” Magnet parser

Imports `containsMagnetLink`, `findMagnetUri`, `extractInfohash`, `isValidHexInfohash`, `isValidBase32Infohash`, `base32ToHex`, `parseMagnetUri`, `buildMagnetUri`, `getMagnetDisplayName` from `../src/parser/magnet`; fixtures from `../fixtures/magnets.json`.

- `containsMagnetLink`: true for text w/ `magnet:?`; false for plain text; false for "";
- `findMagnetUri`: finds all in `fixtures.magnetsInText.text` (count === expectedCount=2); [] for no-magnet text; **deduplicates** identical URIs (same uri twice â†’ length 1); [] for "".
- `extractInfohash`: extracts 40-char hex from `validMagnets[0]` (=== its infohash); null for `http://example.com`; null for `magnet:?dn=test` (no bti); **lowercases** uppercase infohash.
- `isValidHexInfohash`: true for 40-char lower-hex; true for 40-char UPPER-hex; false for 39-char; false for non-hex (gâ€¦); false for "".
- `isValidBase32Infohash`: true for 32-char base32 `MFRGGZDFMZWQ2LKMNZXC4ZTFMRXCXB0`; false for 31-char; false for non-base32 (!â€¦).
- `base32ToHex`: converts `fixtures.base32Infohash.base32` â†’ `.hex` (length 40); throws for "invalid".
- `parseMagnetUri`: parses complete URI (`validMagnets[0]` â†’ infohash/displayName/uri match, `sourceElement===null`); parses trackers (`validMagnets[1]`); multiple trackers (`validMagnets[2]`, length 2); web seeds (`validMagnets[3]`, 1 seed); no display name (`validMagnets[4]` â†’ `displayName===null`); throws for non-magnet; throws for magnet w/o valid infohash

- (magnet:?dn=test); **stores** sourceElement when provided
- (document.createElement("a")); normalizes infohash to lowercase.
- buildMagnetUri: minimal (magnet:?xt=urn:btih:<hash>); w/ display name (&dn=My+File); w/ trackers (url-encoded tr=udp%3A%2F%2F...); throws for invalid infohash.
- getMagnetDisplayName: returns display name when available ("Ubuntu 22.04 LTS"); returns truncated infohash fallback containing "Torrent " and "... " when no display name.

Covers: detection, multi-find + dedup, infohash extraction/validation (hex + base32), base32→hex conversion, full parse (trackers/webseeds/displayName/sourceElement), URI build + url-encoding, display-name fallback. Strong, fixture-driven. **Uses jsdom** document (**in parse-with-element test**).

tests/unit/queue.test.ts (Unit, Jest) ¶ OfflineQueue

Imports OfflineQueue from ../../src/api/queue; type ServerConfig from ../../src/types/config. **Mocks** ../../src/shared/storage via jest.mock ¶ storageGet/storageSet/storageRemove backed by an in-memory mockStorageData object. Defines a full testServer: ServerConfig literal (id/name/url/authMethod:œnoneœ/active/username:null/encrypted*:null/requestTimeout:5000/verifySsl/defaultCategory/defaultSavePath/startPaused/skipHashCheck/contentLayout:œoriginalœ/autoTMM/uploadLimit:0/downloadLimit:0). beforeEach: new OfflineQueue(10) + clear mockStorageData. afterEach: stopAutoProcessing().

- Basic operations: init size 0; enqueue(infohash, magnet, null, displayName, serverId) ¶ size 1, item.torrent.infohash/displayName set, attempts===0; default priority "normal"; explicit priority "high"; dequeue(id) ¶ true + size 0; dequeue("non-existent") ¶ false.
- Queue limits: OfflineQueue(3) with 4 enqueues ¶ size 3 (respects max); OfflineQueue(2) with 3 ¶ keeps 2 newest (items[0].infohash===œ2œ, items[1]===œ3œ ¶ **removes oldest, FIFO eviction**).
- Clear: clear() ¶ size 0.
- Persistence: after enqueue, mockStorageData has keys (storageSet called).
- Auto-processing ¶ partial BLUFF ¶ both end expect(true).toBe(true): start+stop no-error; no duplicate auto-processing (double-start no-throw).
- Queue processing: empty queue ¶ {processed:0, succeeded:0, remaining:0}; after 1 enqueue ¶ processQueue ¶ processed:1, results.length:1 (will fail to send since adapter canœ™t connect, but structure asserted ¶ does NOT assert succeeded/failed values).

Covers: enqueue/dequeue, priority default+explicit, max-size eviction (FIFO), clear, persistence-call, processQueue structure. **Does NOT cover:** retry/attempts increment, exponential backoff, actual success vs failure of processing (only structure), restore-from-storage on construction, concurrent enqueue.

tests/unit/scanner.test.ts (Unit, Jest, jsdom) â€” Scanner system

Imports ScannerOrchestrator from ../../src/scanner/orchestrator;
TypedEventEmitter from ../../src/shared/events; getSiteConfig,
isKnownTorrentSite from ../../src/scanner/site-db.

- TypedEventEmitter (7): on + emit calls listener once; multiple listeners both fire; on returns unsub fn that stops events; once fires only once across 2 emits; listenerCount accurate (0â†’1â†’0 with unsub); emit w/ no listeners doesnâ€™t throw; **listener error isolation** â€” bad listener throws but emit doesnâ€™t throw and good listener still fires.
- ScannerOrchestrator (3): beforeEach clears document.body.innerHTML, constructs orchestrator with {enableLinkScanner:true, enableTextScanner:true, mutationDebounceMs:100, observeMutations:false}; afterEach stop(). Tests: init
getDetectedCount()===0 + hasInitialScanCompleted()===false;
isCurrentlyScanning()===false; getDetectedTorrents()===[].
NOTE: never actually runs a scan â€” only initial-state assertions.
- Site Database (6): recognizes 1337x.to, nyaa.si, yts.mx as known; false for example.com/google.com; getSiteConfig("https://1337x.to") â†’ not null, name==="1337x", domain==="1337x.to", selectors.length>0; null for unknown; **www-prefix matching** (www.1337x.to â†’ domain 1337x.to); 1337x config has a href^="magnet: selector.
- DOM Magnet Link Detection (4): beforeEach clears body. Tests via **raw** document.querySelectorAll (NOT the scanner code) â€” 2 magnet anchors found; 1 .torrent link found (a[href\$=".torrent"]); non-torrent links â†’ 0 magnets; nested magnet anchor found. â€” i, â€” These 4 test jsdom/querySelector, NOT the production scanner â€” weak coverage of actual scan logic.

Covers: event emitter (thorough), site DB lookup + www-matching, orchestrator initial state.

Does NOT cover: actual scanning of DOM (the orchestrator never scans in tests), MutationObserver-driven rescans, debounce behavior, link-scanner/text-scanner real extraction, deduplication of detected torrents.

tests/unit/torrent-file.test.ts (Unit, Jest) â€” .torrent parser

Imports parseTorrentFile, computeInfohash from ../../src/parser/torrent-file; encode from ../../src/parser/bencode. Two helper builders:
createMinimalTorrent() (single-file: announce + info{name,piece length:262144,pieces:Uint8Array(20).fill(0xab),length:1024});
createMultiFileTorrent() (announce + announce-list[2] + creation date + comment + info{name,piece length,pieces.fill(0xcd),files:[{512,[file1.txt]},{1024,[subdir,file2.txt]}]}).

- computeInfohash (5): computes consistent 40-char ^[a-f0-9]{40}\$; different content â†’ different hash; identical content â†’ same hash (deterministic SHA1 of info dict); throws for invalid data ("not a torrent"); throws for torrent without info dict.
- parseTorrentFile (11): minimal single-file (infohash regex, name test-file.txt, pieceLength 262144, numPieces 1, totalSize 1024, files.length 1, files[0].fullPath/length); multi-file (name test-folder, files 2, totalSize 1536=512+1024); extracts tracker URLs (>0, first === announce); multiple trackers from announce-list (2); metadata (creationDate 1700000000, comment â€œTest torrentâ€œ); private flag true when private: 1; defaults private false; throws for non-dictionary (i42e); throws for no info; throws for no name; numPieces calc (20 bytes = 1 piece).

- File information (1): multi-file path building `files[0].path ["file1.txt"] / fullPath "file1.txt"; files[1].path ["subdir", "file2.txt"] / fullPath "subdir/file2.txt".`

Covers: infohash SHA1 computation (determinism, uniqueness, error paths), single+multi file parse, totalSize aggregation, tracker extraction (announce + announce-list), metadata (date/comment), private flag, piece count, nested file path joining, malformed rejection. Strong. Depends on `bencode.encode` (integration-ish but in unit dir).

tests/e2e/content.spec.ts (E2E, Playwright) **Content-script detection**

`test.describe("Content Script Detection").Uses page.setContent(html) + page.waitForTimeout(500). NOTE: does NOT load the extension/content-script asserts on raw DOM via Playwright locators. So it verifies the test page shape, not that the content script ran. (See discrepancy #5 no extension loaded.)`

- detects magnet links on a simulated torrent page `2 magnet anchors + 1 .torrent anchor present(toHaveCount).`
- detects magnet links in text content `body toContainText("magnet:").`
- ignores links inside script tags `magnet inside <script> not counted; only the real anchor (count 1).`
- handles pages with no torrent content `0 magnet anchors.`

Effective coverage: near-zero of actual extension behavior these are DOM-shape sanity checks. Boba MUST rewrite to load the real built content script and assert it injected badges/detected torrents (user-observable).

tests/e2e/options.spec.ts (E2E, Playwright) **Options page UI**

`test.beforeEach: page.goto("chrome-extension://test-id/options/index.html")` (hardcoded id see #5). 16 tests, all asserting DOM structure of the options page:

- loads page with `.sidebar; .sidebar-brand h1 text "BobaLink".`
- has 4 `.nav-items: Servers / General / Advanced / About.`
- `#section-servers` active by default.
- navigate to General `#section-general` active, `#section-servers` not active.
- navigate to Advanced `#section-advanced` active.
- `#btn-add-server` visible + text `Add Server`.
- add-server opens `#server-modal` (visible) + `#modal-title Add Server`.
- server form fields `#server-name/#server-url/#server-auth` visible.
- close modal via `#modal-close` `#server-modal` hidden.
- has `#btn-auto-discover` visible.
- General section toggles `#setting-auto-scan/#setting-highlight/#setting-notifications` visible.
- Advanced section numeric inputs `#setting-health-interval/#setting-max-history/#setting-max-queue` visible.
- About section active + `.about-card` contains `BobaLink`.

Covers (intent): options-page nav, server modal open/close, settings inputs presence. **All hinge on the hardcoded extension URL resolving** non-functional without a fixture (#5). Asserts

- infohash: 00;
- displayName: null;trackers: [].
- invalidMagnets (7 strings â€” note: NOT referenced by any current test, an unused fixture): "not-a-magnet", "magnet:", "magnet:?xt=urn:btih:short", "magnet:?xt=urn:btih:GGGG...GGGG" (40 Gâ€™s, non-hex), "magnet:?xt=urn:btih:1234...5678&dn=" (empty dn), "http://example.com/file.torrent", "ftp://invalid-protocol.example.com".
- magnetsInText:text: "Check out this magnet:?xt=urn:btih:1234567890abcdef1234567890abcdef12345678&dn=Linux+ISO download. Also try magnet:?xt=urn:btih:abcdef1234567890abcdef1234567890abcdef12&dn=Movie for movies.";expectedCount: 2.
- base32Infohash:base32: "MFRGGZDFMZTWQ2LKMNZXC4ZTFMRXCXB0";hex: "68ac55c71d6de1c48abdd2efebf1e7a8d09f1b1e".

Note: invalidMagnets is defined but never asserted in the reference magnet tests â†’ negative-path fixture coverage is left on the table. Bobaâ€™s magnet tests should consume it (each must throw/return null).

Test config extraction

jest.config.ts (unit)

- preset:** ts-jest/presets/default-esm (ESM + TypeScript).
- testEnvironment:** jsdom (overridable per-file via @jest-environment comment).
- roots:** <rootDir>/tests/unit, <rootDir>/src.
- testMatch:** **/tests/unit/**/*.test.ts, **/src/**/*.test.ts (co-located src tests allowed).
- moduleNameMapper:** ^~/ (.*)\$ and ^@/ (.*)\$ â†’ <rootDir>/src/\$1.
- transform:** ^.+\.tsx?\$ via ts-jest { useESM:true, tsconfig: { jsx:"preserve", esModuleInterop:true } }.
- moduleFileExtensions:** ts, tsx, js, jsx, json, node.
- collectCoverageFrom:** src/**/*.ts MINUS *.d.ts, index.ts, assets/**/*.tsx, options/**/*.ts, content/**/*.ts, background/**/*.ts â†’ coverage measured ONLY on pure-logic modules (parser/api/scanner/shared/utils/types).
- coverageThreshold.global:** branches/functions/lines/statements all **80%** (NOT 100% â€” well below Bobaâ€™s mandate).
- coverageReporters:** text, text-summary, lcov, html. **coverageDirectory:** <rootDir>/coverage.
- setupFilesAfterEnv:** <rootDir>/tests/unit/setup.ts.
- setupFiles:** <rootDir>/tests/unit/chrome-mock.ts (runs before framework â€” installs globalThis.chrome).
- clearMocks:** true. **restoreMocks:** true. **testTimeout:** 10000. **verbose:** true. **extensionsToTreatAsEsm:** [".ts"]. **globals.ts-jest.useESM:** true.

playwright.config.ts (e2e)

- testDir:** ./tests/e2e.
- fullyParallel:** false (shared browser state). **workers:** 1. **retries:** CI 2 / local 0. **forbidOnly:** in CI.
- reporter:** html (open never), list, plus github in CI.

- **use:** baseUrl: "chrome-extension://test-id/" (placeholder id), trace: "on-first-retry", screenshot: "only-on-failure", video: "on-first-retry", viewport: 1280~720.
- **use.launchOptions.args:** --disable-extensions-except={EXTENSION_PATH||"./dist"} and --load-extension={EXTENSION_PATH||"./dist"} â€” **extension-loading mechanism: load unpacked from ./dist (or \$EXTENSION_PATH).** (Chromium only â€” Firefox MV3 extension loading via these args does NOT work the same way.)
- **projects:** chromium (devices["Desktop Chrome"], channel:"chromium") AND firefox (devices["Desktop Firefox"]). âšŒ Firefox project would not load the extension via Chromium args â€” effectively broken for extension e2e.
- **webServer:** command:"npm run build",url:"http://localhost:3000" (no such server â€” see #4), reuseExistingServer unless CI, timeout:120000.
- **globalSetup/globalTeardown:** ./tests/e2e/global-setup.ts/global-teardown.ts â€” **MISSING FILES (#4).**
- **timeout:** 30000. **expect.timeout:** 5000.

Extension-loading takeaway: the *intended* mechanism is --load-extension=./dist (built MV3 bundle) under persistent Chromium context. But no fixture captures the assigned extension id and rewrites chrome-extension://test-id/... , so popup/options specs cannot resolve. Bobaâ€™s e2e MUST: build â†’ launch persistent context with --load-extension â†’ read the service-worker/extension id â†’ navigate to the real chrome-extension://<id>/....

COVERAGE MAP (feature/module â†’ reference tests â†’ type â†’ GAPS)

Feature / module	Reference test(s)	Type	What IS covered	GAPS (not covered by reference)
bencode src/ parser/ bencode	bencode.test.ts	unit	encode/decode all 4 types, binary mode, key-sort, hex utils, malformed rejection, torrent round-trip	fuzz/property-based, huge payloads (stress), perf/benchmark, deeply-nested recursion limits (chaos)
magnet parser src/parser/ magnet	magnet.test.ts, fixture magnets.json	unit	detect/find/dedup, infohash hex+base32 validate+convert, parse (trackers/webseeds/dn/sourceElement), build+encode, dn fallback	invalidMagnets fixture unused (7 negatives), malformed-fuzz, perf on huge pages, v2 (btmh/multihash) infohashes
.torrent parser src/parser/ torrent-file	torrent-file.test.ts	unit	infohash SHA1 determinism+uniqueness+errors, single/multi parse, totalSize, trackers, metadata, private, piece count, path join	real-world large .torrent files, hybrid v1+v2, corrupted-file chaos, perf/benchmark on big torrents
API client src/ api/client	api-client.test.ts	unit (fetch mocked)	url normalize, login flow, version, magnet add, GET/	real cookie header propagation (Auth block is a bluff),

Feature / module	Reference test(s)	Type	What IS covered	GAPS (not covered by reference)
			POST, retry-5xx, 429, content-type branching	timeout-exact, abort/cancel, concurrent, integration vs live qBittorrent , auth-method variants (apikey, basic) attempts/retry/backoff, success-vs-failure outcomes, restore-on-construct, concurrency (stress), persistence chaos (corrupt storage)
offline queue src/api/queue	queue.test.ts (storage mocked)	unit	enqueue/dequeue, priority, FIFO eviction, clear, persist-call, processQueue structure	
scanner: event emitter src/shared/events	scanner.test.ts	unit	on/once/unsub, multi-listener, count, error isolation	high-volume emit (perf), reentrancy
scanner: site DB src/scanner/site-db	scanner.test.ts	unit	known-site recognition (1337x/nyaa/yts), config lookup, www-match, selector presence	full site-db coverage (only 3 sites probed), stale-selector detection, regression for each shipped site
scanner: orchestrator src/scanner/orchestrator	scanner.test.ts	unit	initial-state only	actual scanning , mutation-observer rescans, debounce, link/text extraction, dedup “orchestrator never runs a scan
DOM detection (querySelector)	scanner.test.ts (DOM block), content.spec.ts	unit + e2e	querySelector finds magnets/.torrent	tests jsdom/raw DOM, NOT the production content script “real content-script injection unverified does NOT import prod module (bluff) “real
credential crypto src/utls/crypto	crypto.test.ts	unit (self-contained copy)	AES-GCM/PBKDF2 roundtrip, wrong-pw, tamper, IV/salt, perf, object serialize	crypto.ts is UNTESTED; key-rotation, versioned-format migration, storage integration
shared/storage src/shared/storage	(mocked away in queue.test)	“	NONE (always mocked)	direct unit tests of storageGet/Set/Remove, quota handling, chrome.storage errors

Feature / module	Reference test(s)	Type	What IS covered	GAPS (not covered by reference)
shared/errors src/shared/errors	imported by api-client	indirect	error classes constructed	direct error-class tests excluded from coverage; real render of detected torrents, selectâ€™send flow, API call on send, error states
popup UI src/popup	popup.spec.ts	e2e (non-functional)	DOM presence (title/empty-state/buttons)	excluded from coverage; server saveâ€™persist, toggleâ€™setting change, auto-discover behavior, validation
options UI src/options	options.spec.ts	e2e (non-functional)	DOM presence (nav/modal/inputs)	excluded from coverage; real injection, badge rendering, message passing to background
content script src/content	content.spec.ts	e2e (non-functional)	DOM shape only	excluded from coverage; message router, alarms, context-menu handlers, badge updates, install/startup
background/service worker src/background	NONE	â€”	NONE	
types/config src/types/config	used by queue.test	indirect	ServerConfig shape exercised	n/a

Test TYPES the reference suite is MISSING (vs Bobaâ€™™s 100%-coverage mandate)

The reference covers **2 of the mandated types** (unit + e2e), and even those have bluff/non-functional gaps. Per CONST Â§11.4.27 (â€œ100% test-type coverageâ€œ), Â§11.4.25, Â§11.4.85 (stress+chaos), Â§11.4.58/Â§11.4.83 (Challenges + QA evidence), the following MUST be added:

1. **Integration** â€” real cross-module flows: parserâ€™queueâ€™api-clientâ€™live qBittorrent/proxy (Bobaâ€™™s :7186 download proxy / :7187 merge service). The reference mocks fetch and storage everywhere; CONST Â§11.4.27(A) forbids mocks outside unit tests. NEEDED: content-scriptâ€™background message passing, options-saveâ€™storageâ€™api-auth, queue-process against a real WebUI.
2. **E2E (functional, real extension)** â€” load the actual built MV3 bundle, resolve the real extension id, drive the popup/options/content script end-to-end with user-observable

- assertions (torrent rendered, server saved+persisted, sendâ†’torrent appears in qBittorrent). The reference e2e is skeleton/non-functional (hardcoded `test-id`, no extension load fixture, missing global-setup).
3. **Automation / autonomous QA** â€” fully self-driving runs re-runnable NÃ— (CONST Â§11.4.98 / Â§11.4.50), no manual intervention; HelixQA bank entries.
 4. **Security / penetration** â€” credential-leak audits (CONST Â§11.4.10), XSS via injected magnet/dn into popup DOM, CSP of extension pages, permission-scope minimization, MV3 host-permission abuse, malformed-magnet/.torrent injection, cookie/SID handling, encrypted-credential at-rest verification (the crypto module â€” for real, not a copy).
 5. **DDoS / load** â€” many torrents detected on one page (1000s of magnet links), rapid scan cycles, burst of queue enqueues, API request flooding/rate-limit behavior under load.
 6. **Scaling** â€” large .torrent files, large queue (1000s items + persistence), large detected-set rendering in popup, many configured servers.
 7. **Chaos** â€” fault injection: chrome.storage failures/corruption mid-write, network drop/timeout/reorder during queue processing, server-worker termination mid-operation, corrupt persisted state recovery, partial-write faults (CONST Â§11.4.85 closed-set: process-death, network-fault, input-corruption, resource-exhaustion, state-corruption). The current chrome-mock is happy-path only â€” needs an error-injecting variant.
 8. **Stress** â€” sustained load (Nâ‰¥100 iters / â‰¥30s), concurrent contention (Nâ‰¥10 parallel enqueues/scans), boundary inputs (empty/max/off-by-one) with latency p50/p95/p99 captured (CONST Â§11.4.85).
 9. **Performance** â€” only `crypto.test.ts` has perf asserts (and on a copy). NEEDED: parse-time benchmarks (bencode/torrent/magnet on large inputs), scan latency on heavy DOMs, popup render time, with captured evidence.
 10. **Benchmark** â€” formal benchmark suite with baselines + regression detection (separate from inline perf asserts).
 11. **UI tests** â€” beyond DOM-presence: actual rendering correctness, badge counts, list rendering of detected torrents, modal interactions producing state changes (current e2e asserts presence, not behavior).
 12. **UX tests** â€” flows from the end-userâ€™s perspective: detectâ†’selectâ†’send happy path, error feedback, empty/loading/error states, accessibility (a11y), keyboard nav, the configured `chrome.commands` shortcuts.
 13. **Challenges** (`./challenges/scripts/`) â€” CONST Â§11.4.27(B), driving the real built extension against a real qBittorrent + Boba proxy, asserting user-observable outcomes (no false success).
 14. **HelixQA banks + autonomous QA sessions** â€” CONST Â§11.4.27 â€” registered test banks per surface, autonomous session execution with captured evidence per check.
 15. **Anti-bluff remediation across existing types** â€” every test must fail against a no-op stub and assert user-observable outcomes (CONST Â§11.4 / Â§11.4.1). The reference contains explicit bluffs (the `expect(true).toBe(true)` Auth + auto-processing blocks; the crypto copy-not-module; non-functional e2e; dead-assert lines) that violate this â€” they are anti-patterns to fix, not patterns to copy.
-

Open questions (for the plan)

1. **Runner:** standardize on **Vitest** (matches Boba frontend/ + `crypto.test.ts`) or **Jest** (matches the other 6 unit files)? Coverage config and ESM handling differ.
2. **Coverage threshold:** reference enforces **80%** and EXCLUDES all UI/entrypoint dirs. Boba mandates ~100% across types â€” confirm the line/branch threshold AND whether popup/options/content/background must be brought INTO coverage (they must, per Â§11.4.27).

3. **E2E extension-loading fixture:** need a Playwright fixture that builds `./dist`, launches a persistent Chromium context with `--load-extension`, captures the real extension id (via the service worker target), and parameterizes `chrome-extension://<id>/...`. Does Boba target Chromium only, or must Firefox MV3 (different load mechanism) be supported? The reference™s Firefox project is broken for extensions.
4. **Live backends for integration/challenge:** which real services do extension tests run against? qBittorrent WebUI (:7185 via proxy :7186), merge service (:7187), webui-bridge (:7188), boba-jackett (:7189)? CONST Â§11.4.27 requires real services for all non-unit types.
5. `global-setup.ts/global-teardown.ts` referenced by playwright config but absent? define their contract (build extension, start backend containers, seed test data, capture extension id).
6. **Crypto module path:** reference comment says `src/utils/crypto.ts`; `queue/torrent` imports use `src/shared/*`, `src/parser/*`, `src/api/*`. Confirm the production crypto module path Boba will use, and rewrite crypto tests to import it.
7. **Chrome mock error-injection:** chaos/stress need a configurable mock that can throw / return failures / corrupt storage? extend `chrome-mock.ts` or build a separate fault-injecting harness?
8. `invalidMagnets` **fixture:** wire the 7 unused negative cases into magnet tests (each must throw or return null).
9. **Manifest version source:** `chrome-mock` hardcodes `name: "BobaLink"`, `version: "1.0.0"`? does the plan rename to a Boba-branded extension, and should tests read the real manifest?
10. **CI:** Boba CI is manual (`./ci.sh`) and forbids `.github/workflows`. The playwright config™s `reporter: github + forbidOnly: !!CI` assume GitHub Actions? must be reworked for Boba™s manual-CI model.